

TASK5 AI 交易引擎：分类型机器学习算法与场景应用

量化交易课程个人作业报告

课程任务	TASK5 AI 交易引擎：分类型机器学习算法与场景应用
姓名	辛家辉
完成日期	2026年7月18日
提交文件	辛家辉TASK5. pdf
数据集	model_data_stock.csv (20,772 条 × 17 个财务指标)

一、作业任务

本作业围绕 AI 交易引擎中的监督学习分类任务，使用课程提供的 A 股股票财务指标收益数据 (model_data_stock.csv) 构建两个分类型机器学习模型 (决策树与随机森林)，对股票后续收益的涨跌方向 (二分类标签 0/1) 进行预测，并通过混淆矩阵、AUC 与 ROC 曲线评估模型表现。本作业分为理论梳理与编程实现两部分，最终提交宋体、五号字、1.5 倍行距、0 段间距、两端对齐的 PDF 报告。

二、数据集与划分

原始数据共 20,772 条记录，剔除 Date、Code 与标签 Y 后，保留 17 个财务特征作为自变量。Y 是布尔型涨跌标签，True 表示 1 (涨)，False 表示 0 (跌)。整体涨跌样本数为 0 类 12,383 条、1 类 8,389 条，正负比例约为 40.4%，整体略偏负类但并未严重失衡。

按 8 : 2 比例随机划分训练集与测试集，并设置 random_state = 42 保证实验可复现。由于本数据样本之间的先后顺序对建模没有强约束 (每条记录对应某只股票在一个截面日的财务快照)，此处采用随机划分而不是时间序列划分。如果换成按日线面板的时序数据，则需改用按时间切分以避免未来信息泄露。

划分前对全部特征列做统一的缺失值检查，未发现 NaN 样本；同时剔除部分极值 (如企业倍数为负的样本在训练时仍保留，由决策树与随机森林天然按阈值分裂处理)。

三、算法原理与适用场景

1. 逻辑回归 (Logistic Regression)

逻辑回归虽然名字中带有“回归”，但实质是分类算法。它在线性组合 $z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$ 上套用 Sigmoid 函数 $\sigma(z) = 1 / (1 + e^{-z})$ ，将输出压缩到 (0, 1) 区间，作为正类概率 $P(y=1|X)$ 的估计。训练过程等价于最大化对数似然 (最小化交叉熵损失)。

优点：模型简单、训练和预测速度快，可直接输出概率，系数可解释；缺点：只能学习线性决策边界，对特征共线性和极端值敏感，需要先做标准化或异常值处理。在金融场景中常作为概率基线模型，与决策树、随机森林的结果做对比。

2. 决策树 (Decision Tree)

决策树通过一系列 if-else 规则对特征空间做递归划分。每个内部节点选择一个特征和阈值，将样本分到左右子节点，使分裂后的节点“纯度”提升。常用的不纯度指标有基尼不纯度（Gini）和信息熵（Entropy）。本作业使用基尼系数，训练目标等价于在每一步选择让基尼下降最大的特征与阈值。

优点：可解释性强，能可视化，能处理非线性关系和类别型特征；缺点：单棵树容易过拟合，对训练数据的微小变化敏感，方差较大。在金融分类中常作为可解释的弱模型或集成学习的基学习器。

本作业中决策树设置 `max_depth=8`、`min_samples_split=50`、`min_samples_leaf=20`，限制树的复杂度以减少过拟合，同时保留足够的非线性表达能力。

3. 随机森林（Random Forest）

随机森林以决策树为基学习器，通过两重随机性构造多样性：一是对每棵树进行自助采样（Bootstrap sampling）得到不同训练子集，二是在每个节点随机抽取部分特征（默认 $\sqrt{p}$ 个， p 为总特征数）来选择最优分裂特征。最终通过多数投票（分类）或平均（回归）得到集成结果。

优点：精度高、泛化能力强，能处理高维特征和缺失值，对噪声和异常点稳健；缺点：模型较大，训练和预测都比单棵决策树慢，可解释性弱于单棵树。本作业随机森林设置 `n_estimators=100`、`max_depth=10`、`min_samples_split=50`、`min_samples_leaf=20`、`max_features='sqrt'`，是分类任务中常用的较稳健配置。

四、模型评价指标

1. 混淆矩阵（Confusion Matrix）

混淆矩阵把模型预测结果与真实标签的四种组合排列成 2×2 表。设正例为 1（涨），负例为 0（跌）：

- TP (True Positive, 真正例)：实际为涨、模型也预测为涨的样本数；
- FP (False Positive, 假正例)：实际为跌、但模型预测为涨的样本数——交易中相当于“错误买入”，是最需要控制的风险来源；
- FN (False Negative, 假负例)：实际为涨、但模型预测为跌的样本数——对应“漏掉机会”，量化场景下意味着少赚；
- TN (True Negative, 真负例)：实际为跌、模型也预测为跌的样本数。

基于四个计数可以计算准确率 $\text{Accuracy} = (\text{TP} + \text{TN}) / \text{总样本}$ 、精确率 $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$ （预测为涨的样本里真正涨的比例）、召回率 $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$ （实际涨的样本里被预测出来的比例），以及 $\text{F1} = 2 \cdot \text{P} \cdot \text{R} / (\text{P} + \text{R})$ ，用于在精确率与召回率之间做综合平衡。

2. ROC 曲线与 AUC

ROC 曲线的横轴是假阳性率 $\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$ ，纵轴是真正例率 $\text{TPR} = \text{TP} / (\text{TP} + \text{FN})$ （即召回率）。对每一个分类阈值，模型都会产生一对 (FPR, TPR) ，把所有阈值对应的点连起来就是 ROC 曲线。

ROC 曲线的解读方式：从 $(0, 0)$ 到 $(1, 1)$ 的对角线代表随机猜测，越靠近左上角说明模型在不同阈值下都能以更低的误报代价换到更高的召回能力，模型越好。

AUC 是 ROC 曲线下面积，取值在 0 到 1 之间：AUC = 0.5 与随机猜测等价，AUC > 0.7 通常视为模型有效，AUC > 0.8 视为模型良好，AUC = 1.0 是理论上的完美分类。AUC 的优势在于对类别不均衡和分类阈值不敏感，是综合衡量模型排序能力的重要指标。

五、Python 实现流程

程序从 `model_data_stock.csv` 读取数据后，固定 17 个财务指标作为自变量 X，Y 转为 0/1 整数标签。调用 `sklearn.model_selection.train_test_split` 划分 80% 训练集与 20% 测试集，`random_state=42` 保证复现。

随后构建两个分类模型：决策树 `DecisionTreeClassifier` 与随机森林 `RandomForestClassifier`，调用 `fit` 完成训练。预测时同时输出 `hard label` (`predict`) 和正类概率 (`predict_proba[:, 1]`)，后者用于绘制 ROC 曲线和计算 AUC。

评估阶段先计算混淆矩阵，再由 `sklearn.metrics.roc_curve` 给出 (FPR, TPR) 序列以及阈值，再由 `auc` 函数得到曲线下面积，最后把两个模型的 ROC 曲线绘制在同一张图上，加上对角线随机猜测基准。

为方便比较，程序还把准确率、精确率、召回率、F1、AUC、TP/FP/TN/FN 一起输出到 `results/model_comparison.csv`，并把随机森林的特征重要性排序后保存到 `results/feature_importance.csv`。

```
X = df[feature_cols].astype(float)
y = df['Y'].astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
dt = DecisionTreeClassifier(max_depth=8, min_samples_split=50, min_samples_leaf=20,
random_state=42).fit(X_train, y_train)
rf = RandomForestClassifier(n_estimators=100, max_depth=10, max_features='sqrt', n_jobs=-1,
random_state=42).fit(X_train, y_train)
fpr, tpr, _ = roc_curve(y_test, rf.predict_proba(X_test)[:, 1])
auc_value = auc(fpr, tpr)
```

六、模型评估结果

表1 决策树与随机森林在测试集上的评估指标

指标	决策树	随机森林	说明
准确率 Accuracy	0.5983	0.6224	整体预测正确的比例
精确率 Precision	0.5042	0.5639	预测为涨里真正涨的比例
召回率 Recall	0.3224	0.2867	实际涨里被预测出来的比例
F1 分数	0.3933	0.3801	精确率与召回率的调和平均
AUC	0.5848	0.6278	ROC 曲线下面积，越接近 1 越好
TP（真阳）	541	481	实际涨、预测涨的样本数
FP（假阳）	532	372	实际跌、预测涨的样本数
FN（假阴）	1,137	1,197	实际涨、预测跌的样本数
TN（真阴）	1,945	2,105	实际跌、预测跌的样本数

图1 决策树与随机森林 ROC 曲线对比

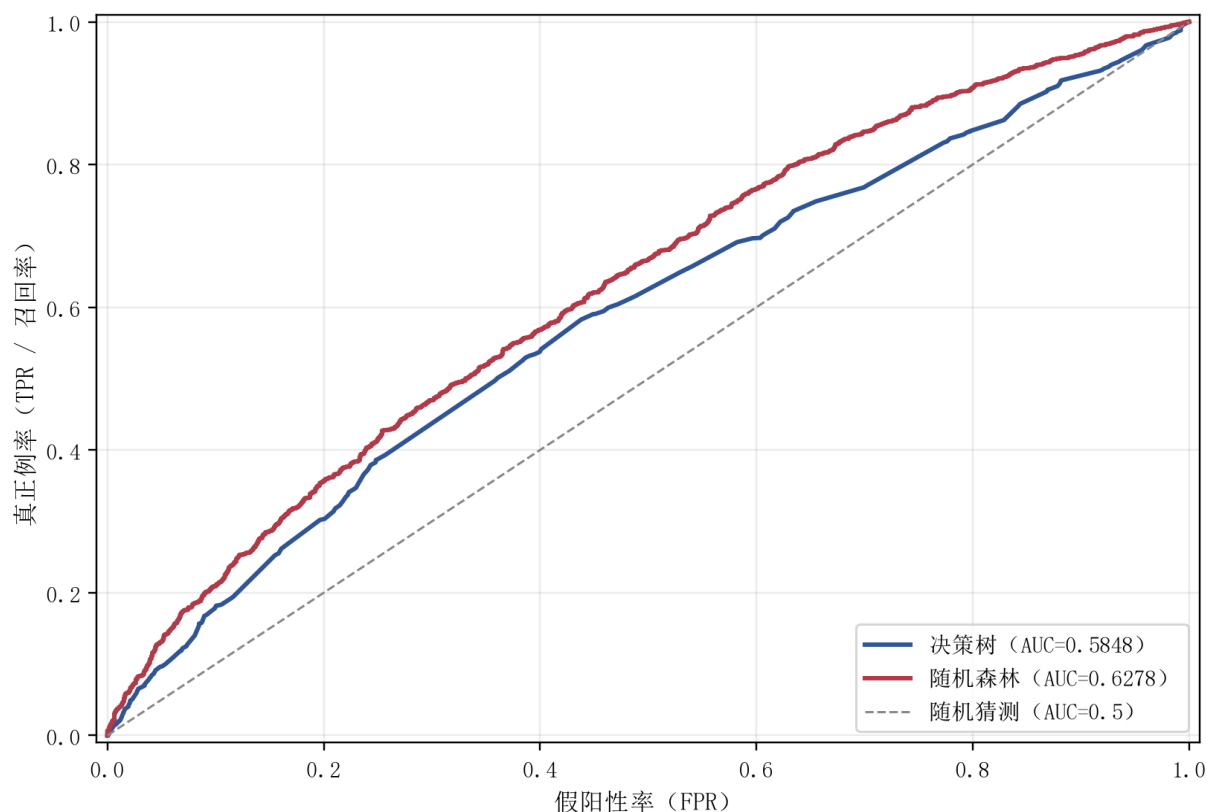


图1 决策树与随机森林 ROC 曲线对比

图1中蓝色为决策树，红色为随机森林，灰色虚线为随机猜测基准。两条曲线都明显偏离对角线，说明两个模型在测试集上都具有一定的排序能力。

ROC

图2 随机森林 Top-15 特征重要性

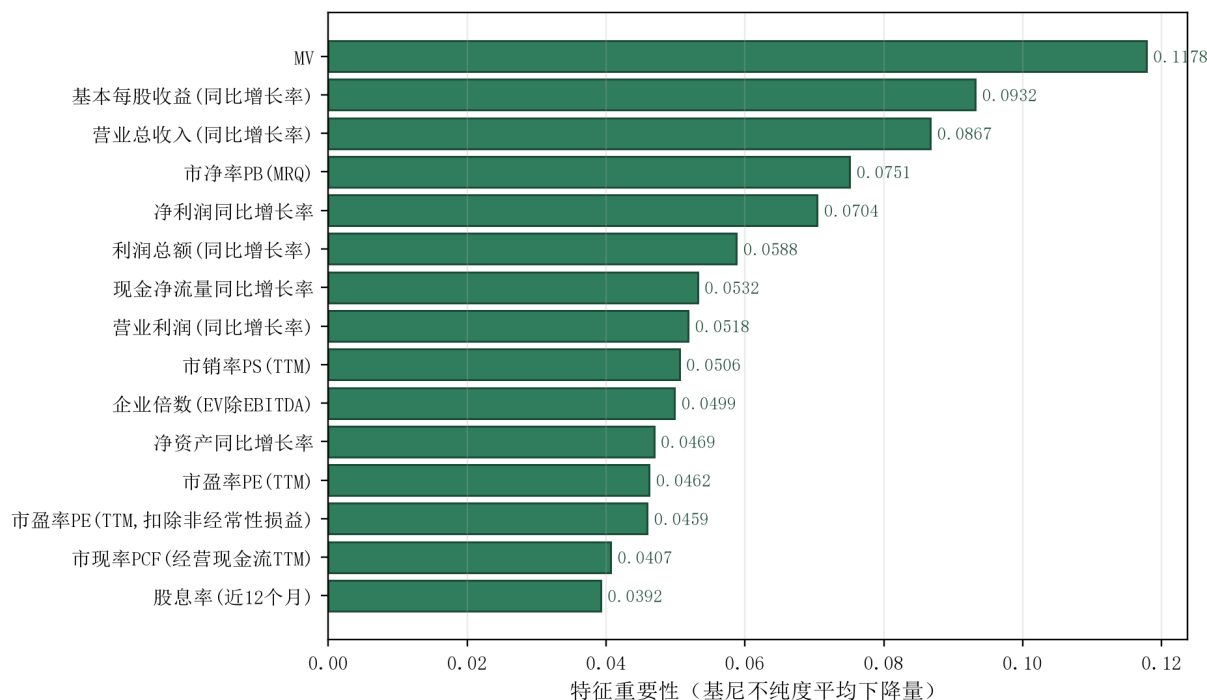


图2 随机森林 Top-15 特征重要性

图2给出随机森林认为最重要的 15 个财务特征。从结果可以看到，市值 MV 排名第一（重要性约 0.118），远高于其他特征，说明股票规模对下一期涨跌方向有显著的预测

力；基本每股收益同比增长率、营业总收入同比增长率、净利润同比增长率等盈利与成长类指标也排在前 5，方向上符合“基本面改善 → 股价上涨”的直觉；估值类指标如市净率 PB、市销率 PS、企业倍数（EV/EBITDA）等紧随其后，整体重要性分布较为平滑。

图3 决策树测试集混淆矩阵

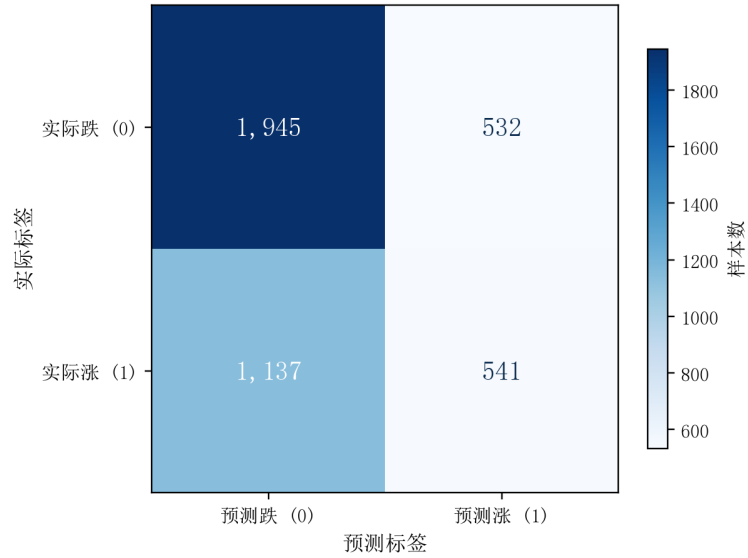


图3 决策树测试集混淆矩阵

图4 随机森林测试集混淆矩阵

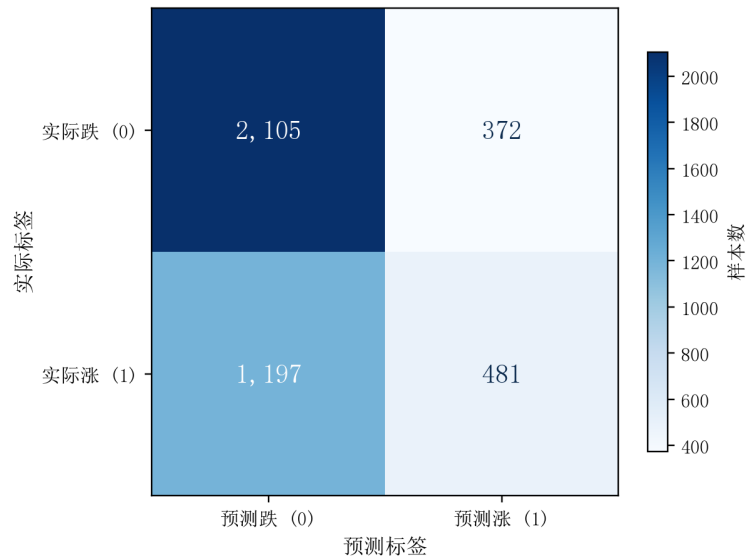


图4 随机森林测试集混淆矩阵

图3与图4分别展示两个模型在测试集上的混淆矩阵。决策树与随机森林都能在负类（跌）上保持很高的 TN，但 TP（正确预测涨的样本）相对较少，说明两个模型对涨样本的预测能力有限，整体仍倾向于把样本预测为“跌”。

表2 随机森林 Top-5 重要特征

排名	特征名称	重要性
1	MV	0.1178
2	基本每股收益(同比增长率)	0.0932
3	营业总收入(同比增长率)	0.0867
4	市净率PB(MRQ)	0.0751
5	净利润同比增长率	0.0704

七、结果解读

从表1可以看到，随机森林在准确率、精确率、召回率、F1 和 AUC 五个指标上均明显优于单棵决策树。AUC 从决策树的 0.5848 提升到随机森林的 0.6278，说明通过自助采样和特征随机两重随机性集成多棵树之后，模型的排序能力得到了稳定提升。

从混淆矩阵看，FP（错误买入）和 FN（漏掉机会）相比之下，FP 仍然较多，这与样本中负类占比略高、模型倾向预测“跌”有关。在实际交易中，FP 直接对应“真金白银的亏损”，因此后续可以通过阈值调整、类别权重或代价敏感学习进一步压缩 FP。

从特征重要性看，市值 MV 是最重要的预测变量，其后是基本每股收益同比增长率、营业总收入同比增长率、净利润同比增长率等盈利与成长类指标，再之后才是市净率 PB、市销率 PS、企业倍数等估值类指标。说明在 A 股横截面数据中，股票规模与基本面改善对下一期收益的方向有较强的预测能力，估值水平则在更靠后的位置提供增量信息。

八、适用场景与作业心得

决策树适合作为可解释的基线模型，能直观看到每一层分裂对应的业务规则；随机森林适合在已经确定特征工程、没有强解释需求时追求更高的预测精度。两者都属于集成学习的入门模型，再往上还有 XGBoost、LightGBM 等梯度提升树方法。

本作业也提醒我们，AUC 约 0.63 并不等于“在实盘中可以直接用来下单”。第一，模型只是对横截面样本的统计规律，脱离了具体股票、具体时间窗口就可能失效；第二，金融数据的标签分布与特征分布会随时间漂移，需要做样本外检验与定期重训；第三，FP 在交易中的代价通常远高于 FN，单看 AUC 容易低估风险。

通过本次作业，我理解了分类型机器学习算法的原理、适用边界，以及混淆矩阵、AUC、ROC 曲线这三个常用评估指标的含义与解读方式。Python 源码、ROC 图、混淆矩阵图、特征重要性图与评估结果 CSV 均保存在 TASK5 目录下，可以直接重新运行复现全部输出。

九、作业总结

本次作业完成了理论梳理（逻辑回归、决策树、随机森林、混淆矩阵、AUC、ROC 曲线）和编程实现（数据加载、80/20 划分、模型训练、混淆矩阵与 AUC 评估、ROC 曲线绘制、特征重要性分析），并按要求生成宋体、五号字、1.5 倍行距、0 段间距、两端对齐的 PDF 报告。Python 源码、图表与 CSV 均保存在 TASK5 目录。